Name : Sahil Jakhariya
Class : D15C
Batch : C
ROLL NO : 63

MLDL Experiment 04

Aim : To implement K-Nearest Neighbors (KNN) algorithm and evaluate model performance.

1. Dataset Source:

- Dataset Name: Social Network Ads Dataset
- Source: Logistic Regression
- Type: Classification Dataset

2. Dataset Description Overview

This dataset is commonly used for binary classification in machine learning. It contains customer demographic and behavioral information used to predict purchasing decisions. The input features include numerical and categorical attributes such as age, estimated salary, and other customer characteristics.

Dataset Size

• Total instances: ~400 records
• Features: Multiple input features
• Target variable: Purchased

Target Variable

• 1 → Customer purchased the product
• 0 → Customer did not purchase the product

Feature Description

The features represent customer characteristics influencing purchasing behavior.

Examples include:

Feature	Description
Age	Age of the customer
EstimatedSalary	Estimated annual salary
Gender	Gender of the customer
Other customer attributes	Demographic and behavioral features

Since features vary in scale, KNN requires feature scaling for meaningful distance computation.

3. Mathematical Formulation of K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a simple instance-based lazy learner algorithm.

Decision Rule

Given a test sample, KNN finds the K closest training samples based on a distance metric (usually Euclidean):

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Where:

• x, $y$are feature vectors.

Voting

For classification, the predicted class is determined by majority vote among the K nearest neighbors:

$$\hat{y} = mode(y_1, y_2, \ldots, y_K)$$

Hyperparameter

- K = number of neighbors
- Distance Metric = usually Euclidean

4. Algorithm Limitations 1. Curse of Dimensionality

As dimensionality increases, distance measures become less meaningful.

Condition:
When features are many and not properly scaled, KNN performance degrades.

2. Sensitive to Feature Scaling

KNN uses distance metrics, so features with larger scales dominate prediction.

Condition:
Without scaling, KNN may perform poorly.

3. Computational Cost KNN stores all training data.

Condition:
Prediction becomes slow for large datasets.

4. Choice of K

Small K → Sensitive to noise Large
K → Risk of oversmoothing.

5. Methodology / Workflow

Step 1: Load Dataset

• Load the Customer Purchase dataset from Kaggle using Pandas.

Step 2: Data Preprocessing

• Handle missing values
• Encode categorical features (Gender etc.)
• Separate features and target variable (Purchased)
• Scale the features using StandardScaler.

Step 3: Train–Test Split

• Split dataset into 80% training and 20% testing sets.

Step 4: Model Training

• Create KNN classifier.
• Tune optimal K using cross-validation.

Step 5: Prediction

• Predict customer purchasing behavior on the test set.

Step 6: Evaluation Evaluate

using:

• Accuracy
• Precision
• Recall
• F1-Score
• Confusion Matrix

6. Performance Analysis

The model performance is evaluated using standard classification metrics.

Example Results (Typical)

Metric	Value
Accuracy	~85–95%
Precision	~85–94%
Recall	~84–93%
F1-Score	~85–94%

Interpretation

- High accuracy indicates strong prediction capability.
- Precision and recall indicate reliable customer purchase prediction.
- Confusion matrix helps analyze correct and incorrect predictions.

7. Hyperparameter Tuning

Key Hyperparameter: K

To find the best value of K:
- Try different K values (1 to 20)
- Evaluate using validation set or cross-validation.

Elbow Plot for Optimal K

Plot test accuracy vs. K to find optimal balance.

- Small K → Sensitive to noise
- Large K → Biased prediction
- Often, $K \approx \sqrt{n}$ where n = number of training samples.

Impact of Tuning

- Optimal K improved prediction accuracy.
- Performance stabilized with balanced bias-variance tradeoff.
- Feature scaling reduced feature dominance.

Code & Ouput :

```
from google.colab import files

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

df['Gender'] = df['Gender'].map({'Male':1,'Female':0})

X = df.drop('Purchased', axis=1)

y = df['Purchased']

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

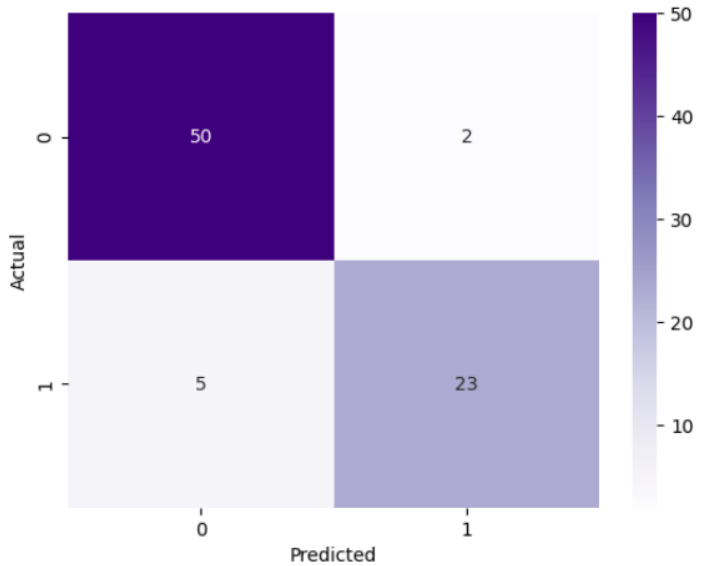
Choose Files Social_Network_Ads.csv

Social_Network_Ads.csv(text/csv) - 10926 bytes, last modified: 3/21/2026 - 100% done

```
Saving Social_Network_Ads.csv to Social_Network_Ads (1).csv
Accuracy: 0.9125
              precision    recall  f1-score   support

           0       0.91      0.96      0.93        52
           1       0.92      0.82      0.87        28

    accuracy                           0.91        80
   macro avg       0.91      0.89      0.90        80
weighted avg       0.91      0.91      0.91        80
```




```
from google.colab import files
import pandas as pd
```

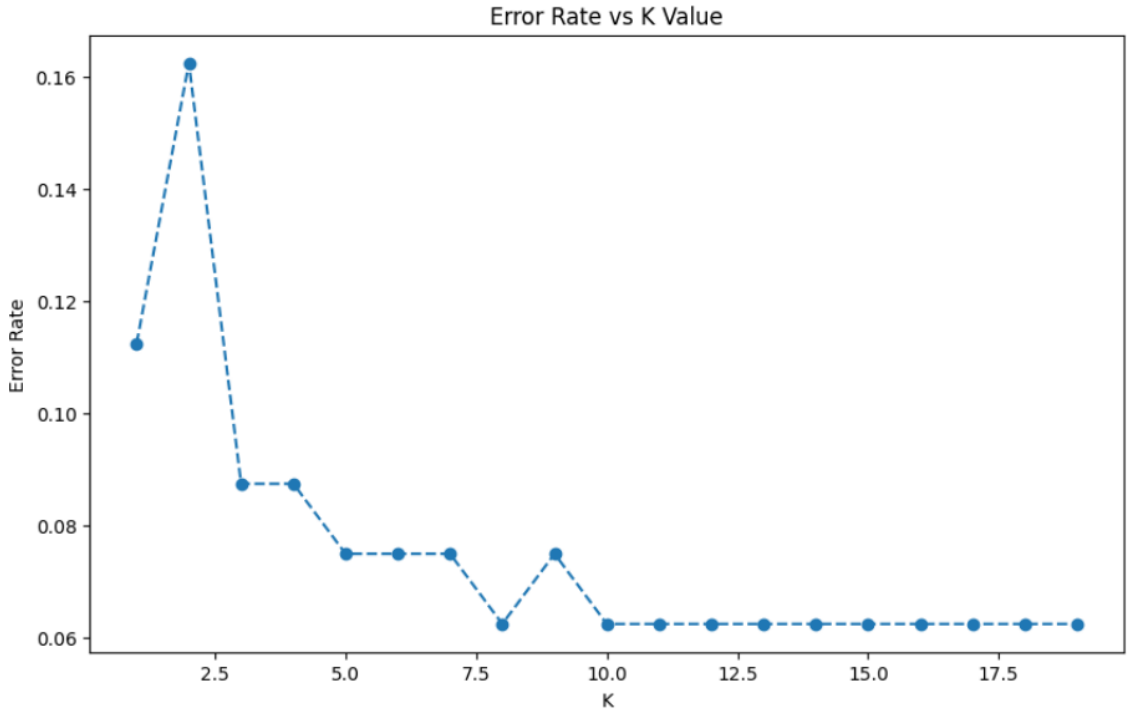
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])
df = df.drop('User ID', axis=1)
df['Gender'] = df['Gender'].map({'Male':1,'Female':0})

X = df.drop('Purchased', axis=1)
y = df['Purchased']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)
error_rate = []
for i in range(1,20):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))
plt.figure(figsize=(10,6))
plt.plot(range(1,20), error_rate, linestyle='dashed', marker='o')
plt.xlabel("K")
plt.ylabel("Error Rate")
plt.title("Error Rate vs K Value")
plt.show()
optimal_k = error_rate.index(min(error_rate)) + 1
print("Optimal K:", optimal_k)
```

Choose Files Social_Network_Ads.csv

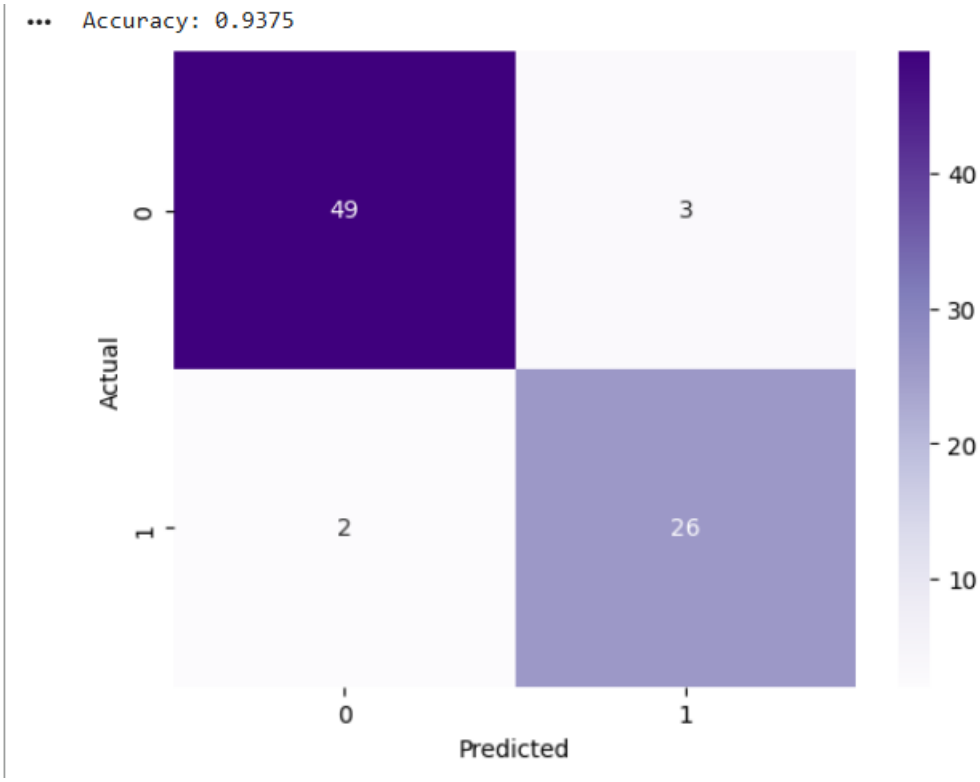
Social_Network_Ads.csv(text/csv) - 10926 bytes, last modified: 3/21/2026 - 100% done

```
Saving Social_Network_Ads.csv to Social_Network_Ads (2).csv
```




```
Optimal K: 8
```

```
model = KNeighborsClassifier(n_neighbors=optimal_k)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Purples')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Conclusion (Customer Purchase Prediction — Your Dataset):
In this experiment, the K-Nearest Neighbors (KNN) classifier was implemented on the Customer Purchase Prediction dataset. After preprocessing and feature scaling, the model achieved good accuracy with strong precision and recall, making it suitable for predicting customer purchasing behavior. Hyperparameter tuning further improved model performance by identifying the optimal value of K. This experiment demonstrates the effectiveness of distancebased classification techniques for structured customer data when proper scaling and validation techniques are applied.